Homework 6 Solution

This notebook solves kernel regression with batch gradient descent (Problem 1), kernel logistic regression on the half-moon data (Problem 2), and answers the conceptual questions on kernels and support vector machines (Problem 3).

```
import os
for v in
["OMP_NUM_THREADS","OPENBLAS_NUM_THREADS","VECLIB_MAXIMUM_THREADS","MKL_NUM_THREADS"]:
    os.environ[v] = "1"   # small matrices: avoid BLAS thread oversubscription
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
DATA = '/Users/xxie/Documents/teaching/BDA411:511/homework/hwk6_data/'
plt.rcParams.update({'font.size': 11})
```

Problem 1: RBF kernel regression with batch gradient descent

Model

Given inputs x_i and targets y_i ($i = 1, ..., n$), we use the Gaussian (RBF) kernel

$$k(x, x') = \exp\left(-\frac{(x - x')^2}{2\sigma^2}\right), \qquad K_{ij} = k\left(x_i, x_j\right),$$

and the model $f(x) = \sum_j a_j k\left(x_j, x\right)$, i.e. $\mathbf{f} = K\mathbf{a}$. The loss is the regularized mean squared error

$$J(\mathbf{a}) = \frac{1}{2n} \| K\mathbf{a} - \mathbf{y} \|^2 + \frac{\lambda}{2}\mathbf{a}^\top K\mathbf{a}, \qquad \nabla J = K(\frac{1}{n}\left(K\mathbf{a} - \mathbf{y}\right) + \lambda\mathbf{a}).$$

Batch gradient descent updates $\mathbf{a} \leftarrow \mathbf{a} - \alpha\nabla J(\mathbf{a})$ using all data at each step. The Hessian is $K(K/n + \lambda I)$, so the loss is smooth with constant $L = \lambda_{\max}(K)\left(\lambda_{\max}(K)/n + \lambda\right)$ and BGD converges for $\alpha < 2/L$. We pick $\alpha = 1/L$. Since K has many tiny eigenvalues, the small ridge λ keeps the problem well posed, and the comparison with the closed-form solution $\mathbf{a}^* = (K + n\lambda I)^{-1}\mathbf{y}$ checks convergence.

```
x = np.load(DATA + 'NonRegression_X.npy'); y = np.load(DATA + 'NonRegression_Y.npy')
n = len(x); print('n =', n, ' x range', x.min().round(2), x.max().round(2), ' y range',
y.min().round(2), y.max().round(2))

def rbf(A, B, sigma):
    return np.exp(-(A[:, None] - B[None, :])**2 / (2 * sigma**2))

sigma, lam = 0.5, 1e-3
K = rbf(x, x, sigma)
lmax = np.linalg.eigvalsh(K).max()
L = lmax * (lmax / n + lam)
alpha = 1.0 / L
print(f'sigma={sigma}, lambda={lam}, lambda_max(K)={lmax:.2f}, L={L:.2f}, alpha=1/L=
{alpha:.4f}')

def J(a): return 0.5/n*np.sum((K@a - y)**2) + 0.5*lam*a@K@a
a = np.zeros(n); hist_J, hist_mse = [], []
iters = 20000
for t in range(iters):
    hist_J.append(J(a)); hist_mse.append(np.mean((K@a - y)**2))
    a -= alpha * K @ ((K@a - y)/n + lam*a)
a_star = np.linalg.solve(K + n*lam*np.eye(n), y)
print(f'final loss {J(a):.5f}, loss at closed-form optimum {J(a_star):.5f}, train MSE
{hist_mse[-1]:.5f}')
print('max |f_BGD - f_closed-form| on training points:', np.abs(K@a - K@a_star).max())
```

```
n = 100  x range 0.03 5.92  y range -1.11 1.2
sigma=0.5, lambda=0.001, lambda_max(K)=21.50, L=4.65, alpha=1/L=0.2153
```

```
final loss 0.00529, loss at closed-form optimum 0.00526, train MSE 0.00771
max |f_BGD - f_closed-form| on training points: 0.015664883261003237
```

```
xg = np.linspace(x.min()-0.2, x.max()+0.2, 400)
fg = rbf(xg, x, sigma) @ a
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
ax[0].scatter(x, y, s=18, marker='o', color='k', label='data')
ax[0].plot(xg, fg, '-', lw=2, label='BGD kernel regression')
ax[0].plot(xg, rbf(xg, x, sigma) @ a_star, '--', lw=1.5, label='closed-form optimum')
ax[0].set_xlabel('x'); ax[0].set_ylabel('y'); ax[0].set_title(r'RBF regression fit
($\sigma=0.5$)'); ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].loglog(np.arange(1, iters+1), hist_J, '-', label='regularized loss J')
ax[1].loglog(np.arange(1, iters+1), hist_mse, '--', label='training MSE')
ax[1].set_xlabel('iteration'); ax[1].set_ylabel('error'); ax[1].set_title('Error decay of
batch gradient descent'); ax[1].legend(); ax[1].grid(alpha=.3, which='both')
plt.tight_layout(); plt.show()
```

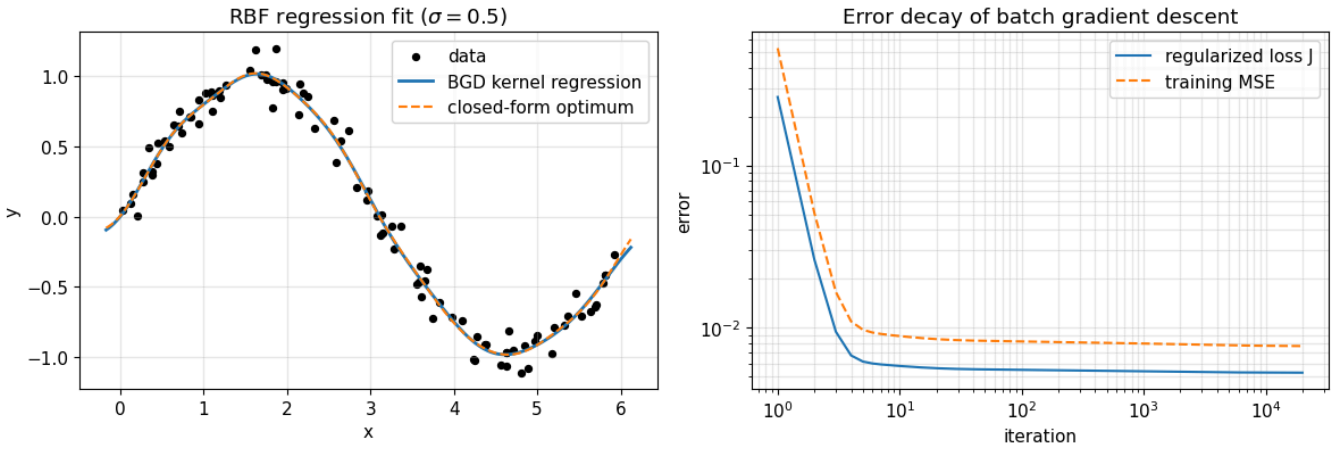


Figure description. Left: scatter plot of the 100 data points (black circles) following a smooth oscillating curve, with the BGD fitted curve (solid) and the closed-form optimum (dashed) lying on top of each other and passing through the data. Right: log-log plot of the regularized loss (solid) and training mean squared error (dashed) against iteration; both decrease rapidly at first and then flatten near a small plateau.

Effect of the bandwidth σ

A small σ makes the kernel local and the fit wiggly (overfitting); a large σ gives a very smooth fit that can underfit. We rerun BGD for three values (each with its own $\alpha = 1/L$).

```
fig, axs = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
for ax, sg, mk in zip(axs, [0.05, 0.5, 3.0], ['o', 's', '^']):
    Kk = rbf(x, x, sg); lm = np.linalg.eigvalsh(Kk).max(); step = 1/(lm*(lm/n + lam))
    aa = np.zeros(n)
    for t in range(20000): aa -= step * Kk @ ((Kk@aa - y)/n + lam*aa)
    ax.scatter(x, y, s=14, marker=mk, color='k', label='data')
    ax.plot(xg, rbf(xg, x, sg) @ aa, lw=2, label='fit')
    ax.set_title(f'sigma = {sg}  (train MSE {np.mean((Kk@aa-y)**2):.4f})');
ax.set_xlabel('x'); ax.grid(alpha=.3); ax.legend()
axs[0].set_ylabel('y'); plt.tight_layout(); plt.show()
```

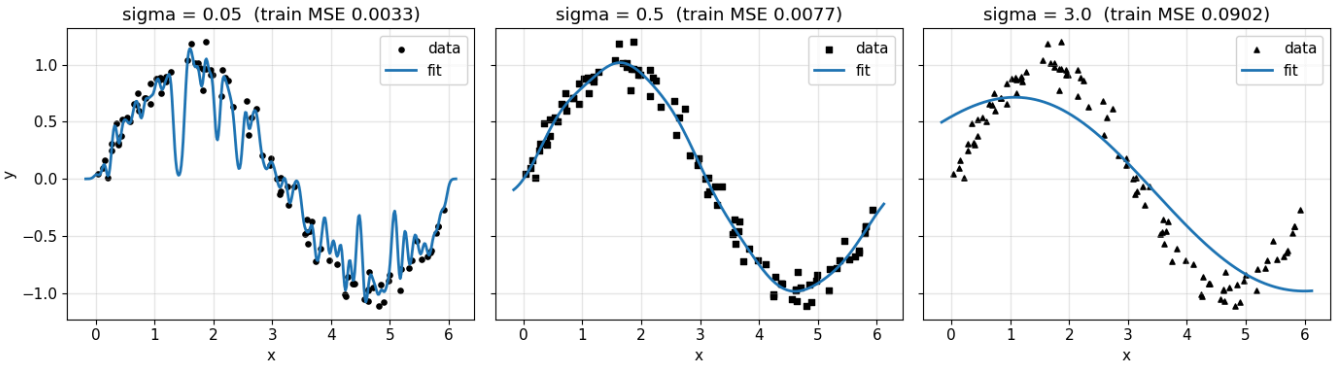


Figure description. Three side-by-side scatter plus line plots of the same data with fits for $\sigma = 0.05$, 0.5 and 3. With $\sigma = 0.05$ the fit follows individual points and is jagged between them, and it decays toward zero away from the data. With $\sigma = 0.5$ it is a smooth curve through the data. With $\sigma = 3$ the curve is too smooth and misses the oscillations, with the largest error.

Problem 2: Kernel logistic regression on the half-moon data

Model

With labels $y_i \in \{-1, +1\}$ and $\mathbf{f} = K\mathbf{a}$, kernel logistic regression minimizes

$$J(\mathbf{a}) = \frac{1}{n}\sum_i \log\left(1 + e^{-y_i f_i}\right) + \frac{\lambda}{2}\mathbf{a}^\top K\mathbf{a}, \qquad \nabla J = K\left(-\frac{1}{n}\mathbf{y} \odot \sigma\left(-\mathbf{y} \odot \mathbf{f}\right) + \lambda\mathbf{a}\right),$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. The predicted class is $\operatorname{sign}(f(x))$ and the decision boundary is $f(x) = 0$. The Hessian is bounded by $K(K/(4n) + \lambda I)$, which gives a safe step $\alpha = 1/L$ with $L = \lambda_{\max}(K)\left(\lambda_{\max}(K)/(4n) + \lambda\right)$. We train with BGD using an RBF kernel $k(x, x') = \exp\left(-\|x - x'\|^2 / \left(2\sigma^2\right)\right)$ and, for comparison, a cubic polynomial kernel $k(x, x') = \left(1 + x^\top x'\right)^3$.

```
X = np.load(DATA + 'half_moon_features.npy'); yl = np.load(DATA +
'half_moon_classlabels.npy')
print(X.shape, np.unique(yl, return_counts=True))
n = len(yl)

def rbf2(A, B, sigma):
    d2 = np.maximum((A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2*A@B.T, 0)
    return np.exp(-d2 / (2*sigma**2))
def poly(A, B, deg=3): return (1 + A @ B.T)**deg

def klr_fit(K, y, lam, iters):
    n = len(y); lm = np.linalg.eigvalsh(K).max(); alpha = 1/(lm*(lm/(4*n) + lam))
    a = np.zeros(n); losses, accs = [], []
    for t in range(iters):
        f = K @ a
        losses.append(np.mean(np.logaddexp(0, -y*f)) + 0.5*lam*a@f)
        accs.append(np.mean(np.sign(f) == y))
        g = K @ (-y/(1+np.exp(y*f))/n + lam*a)
        a -= alpha*g
    return a, np.array(losses), np.array(accs), alpha

lam = 1e-3
K_rbf = rbf2(X, X, 0.5); a_rbf, loss_rbf, acc_rbf, al_rbf = klr_fit(K_rbf, yl, lam, 5000)
K_pol = poly(X, X);      a_pol, loss_pol, acc_pol, al_pol = klr_fit(K_pol, yl, lam, 5000)
print(f'RBF (sigma=0.5): alpha={al_rbf:.4f}, final loss {loss_rbf[-1]:.4f}, train accuracy
{acc_rbf[-1]:.3f}')
print(f'Cubic polynomial: alpha={al_pol:.2e}, final loss {loss_pol[-1]:.4f}, train accuracy
{acc_pol[-1]:.3f}')
```

```
(300, 2) (array([-1,  1]), array([150, 150]))
```

```
RBF (sigma=0.5): alpha=0.2470, final loss 0.1089, train accuracy 0.993
Cubic polynomial: alpha=4.33e-05, final loss 0.2551, train accuracy 0.873
```

```
g1 = np.linspace(X[:,0].min()-.4, X[:,0].max()+.4, 150); g2 = np.linspace(X[:,1].min()-.4,
X[:,1].max()+.4, 150)
G1, G2 = np.meshgrid(g1, g2); grid = np.c_[G1.ravel(), G2.ravel()]
fr = (rbf2(grid, X, 0.5) @ a_rbf).reshape(G1.shape)
fp = (poly(grid, X) @ a_pol).reshape(G1.shape)
fig, axs = plt.subplots(1, 2, figsize=(12, 4.8))
for ax, f, name, acc in [(axs[0], fr, 'RBF kernel', acc_rbf[-1]), (axs[1], fp, 'Cubic
polynomial kernel', acc_pol[-1])]:
    cs = ax.contourf(G1, G2, f, levels=np.linspace(-np.abs(f).max(), np.abs(f).max(), 21),
cmap='RdBu', alpha=.35)
    ax.contour(G1, G2, f, levels=[0], colors='k', linewidths=2)
    ax.scatter(X[yl==-1,0], X[yl==-1,1], c='red', marker='o', edgecolor='k', s=22,
label='Class -1')
    ax.scatter(X[yl==1,0], X[yl==1,1], c='blue', marker='^', edgecolor='k', s=26,
label='Class 1')
    ax.set_title(f'{name}: decision boundary (train acc. {acc:.1%})');
ax.set_xlabel('feature 1'); ax.set_ylabel('feature 2'); ax.legend(loc='upper right')
    fig.colorbar(cs, ax=ax, label='f(x)')
plt.tight_layout(); plt.show()
```

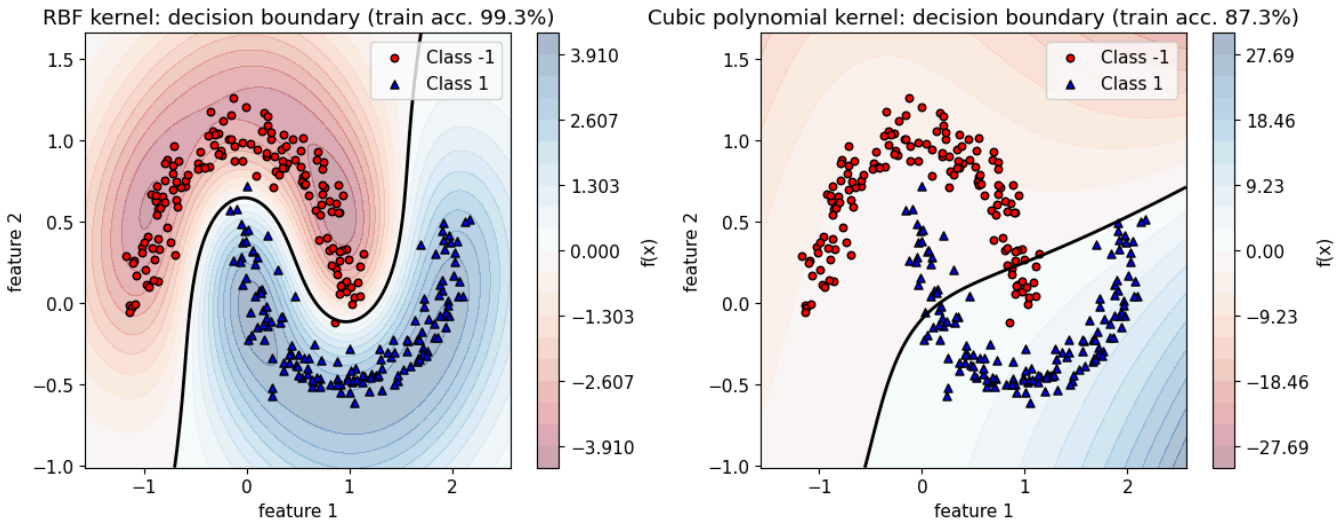


Figure description. Two panels with the half-moon data: red circles for class -1 and blue triangles for class $+1$, over a shaded map of the learned score $f(x)$ with the black decision boundary $f = 0$. In the RBF panel the boundary curves smoothly between the two interlocking moons and separates them almost perfectly. In the cubic polynomial panel the boundary is a smoother global curve that cannot follow the moons and misclassifies noticeably more points (about 13 percent).

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].semilogy(loss_rbf, '-', label='RBF kernel'); ax[0].semilogy(loss_pol, '--',
label='cubic polynomial kernel')
ax[0].set_xlabel('iteration'); ax[0].set_ylabel('regularized logistic loss');
ax[0].set_title('Loss decay of BGD'); ax[0].legend(); ax[0].grid(alpha=.3, which='both')
ax[1].plot(1-acc_rbf, '-', label='RBF kernel'); ax[1].plot(1-acc_pol, '--', label='cubic
polynomial kernel')
ax[1].set_xscale('log'); ax[1].set_xlabel('iteration'); ax[1].set_ylabel('training
misclassification rate'); ax[1].set_title('0-1 error during training'); ax[1].legend();
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

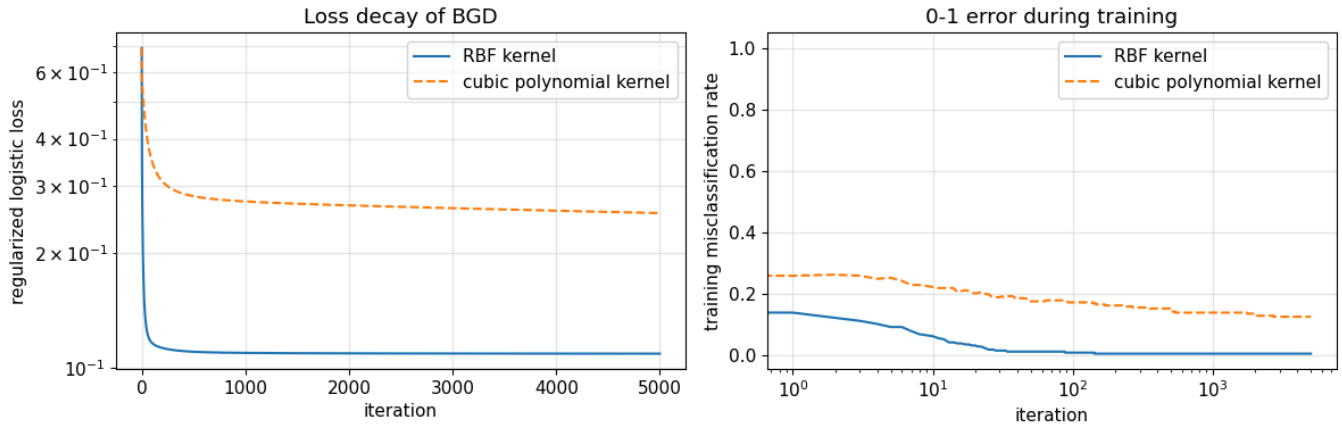


Figure description. Left: semilog plot of the regularized logistic loss against iteration for the RBF kernel (solid) and the polynomial kernel (dashed); both curves decrease monotonically from $\log 2 \approx 0.69$; the RBF curve reaches a much lower value (about 0.11) than the polynomial curve (about 0.26), which is still decreasing slowly because the ill-conditioned polynomial kernel forces a very small step. Right: training misclassification rate against iteration on a log axis for the two kernels, falling to about 1 percent for the RBF kernel and staying near 13 percent for the polynomial kernel.

Comments

Both fits are obtained with plain batch gradient descent with step $1/L$, so the loss decreases monotonically. The RBF kernel gives a flexible local model, which suits the curved class boundary, and it achieves the lowest loss and error. The polynomial kernel is a global model and is less flexible here. The ridge term λ prevents the coefficients from growing without bound on (almost) separable data; σ and λ would be chosen by cross-validation in practice.

Problem 3: Conceptual questions

Why do we use kernel methods?

Linear models cannot represent nonlinear structure. A kernel $k(x, x') = \langle \phi(x), \phi(x') \rangle$ computes inner products in a (possibly infinite-dimensional) feature space without ever forming $\phi(x)$ (the kernel trick). Any algorithm that depends on the data only through inner products, such as ridge regression, logistic regression, SVM or PCA, can then learn nonlinear functions at the cost of working with the $n \times n$ kernel matrix instead of explicit features.

What is the key idea of the support vector machine?

Among all hyperplanes $w^\top x + b = 0$ that separate the classes, choose the one with the maximum margin, the largest distance $2/\|w\|$ to the closest points of each class. This is the convex problem $\min_{w,b} \frac{1}{2}\|w\|^2$ subject to $y_i(w^\top x_i + b) \geq 1$. The solution depends only on the points on the margin, the support vectors, which have nonzero dual variables. A wide margin gives good generalization, and kernels give nonlinear boundaries.

Hard-margin versus soft-margin SVM

Hard-margin SVM requires every point to be classified correctly and outside the margin, $y_i\left(w^\top x_i + b\right) \geq 1$. It exists only for linearly separable data and is sensitive to outliers and noise. Soft-margin SVM allows violations through slack variables,

$$\min_{w,b,\xi} \ \frac{1}{2} \parallel w \parallel^2 \ + C\sum_i \xi_i \quad \text{s.t.} \quad y_i\left(w^\top x_i + b\right) \geq 1 - \xi_i,\ \xi_i \geq 0,$$

and the parameter C trades margin width against training violations (large C approaches the hard margin; small C tolerates more errors).

Slack variable and hinge loss versus 0-1 loss

The slack variable $\xi_i \geq 0$ measures how far point i violates the margin: $\xi_i = 0$ if it is correctly classified outside the margin, $0 < \xi_i \leq 1$ if it is inside the margin but on the correct side, and $\xi_i > 1$ if it is misclassified. Eliminating it gives the hinge loss $\ell_{\text{hinge}} \ = \ \max\left(0,\, 1 - y_i f\left(x_i\right)\right)$. The 0-1 loss $\ell_{\text{0-1}} \ = \mathbf{1}\left[y_i f\left(x_i\right) \leq 0\right]$ counts mistakes directly but is discontinuous and non-convex, so it is NP-hard to minimize. The hinge loss is a convex upper bound on it that can be minimized efficiently. (The problem's phrase "hinge-loss (0-1 loss)" treats the two as related, but they are different losses.)

SVM versus logistic regression

Both are linear classifiers in feature space (and both can be kernelized) and differ in their loss. Logistic regression minimizes the log loss $\log\left(1 + e^{-yf}\right)$, which is smooth, never exactly zero, so every point influences the solution, and it outputs probabilities. SVM minimizes the hinge loss with an ℓ_2 penalty. Points beyond the margin contribute nothing, so the solution is sparse in the support vectors, and it outputs scores and not probabilities. In practice they perform similarly. Logistic regression is preferable when calibrated probabilities are needed, while kernel SVM is often more efficient at prediction time owing to its sparse solution.